Behavior–Realization Separation for Constrained Physical Human–Robot Interaction

Yongyan Cao

***Abstract*—Physical human–robot interaction software often couples desired-behavior specification with constrained realization; we treat these as separate layers. A *behavior layer* supplies a desired contact-port acceleration $a_k^{\mathrm{id}} = f_\theta(e_k, \dot{e}_k, F_{h,k})$. A *realization layer* converts it into constrained robot commands and reports total desired-versus-realized acceleration error instead of hiding it in saturation. A same-objective unconstrained counterfactual separates regularization from constraint intervention, while plant data expose model error. This paper implements a receding-horizon quadratic program realizing memoryless affine behaviors. Changing the behavior modifies objective coefficients through (C_θ, G_θ) while the robot-command variable and feasible set remain unchanged. A planar study instantiates impedance and admittance; the same running layer accepts an impedance–admittance–impedance reassignment without reconstruction, under its existing rate limit. On a torque-controlled 7-DOF Franka FR3 in MuJoCo, the runtime freezes task-space dynamics per solve and enforces torque feasibility across its horizon. Under a sustained 20 N push, it holds a slack-relaxed workspace boundary to within approximately 0.1–0.2 mm, versus 4.4 cm (impedance) and 4.7 cm (admittance) overshoot from instantaneous clipping. A derated actuator budget then activates the torque constraint: horizon-wide enforcement keeps its frozen-model plan feasible to 2.1×10^{-4} N·m, whereas a first-step-only ablation plans up to 11.329 N·m beyond budget; on the executed nonlinear plant, where both share the same local-model error, the gap is smaller but still favors horizon-wide enforcement (0.161 vs. 0.380 N·m). These results are a focused demonstration of behavior–realization separation on an executable interface (scope in Section VI-E).**

***Index Terms*—Physical human–robot interaction, behavior–realization separation, robot-control architecture, predictive realization, model predictive control, impedance control, admittance control.**

I. Introduction

Interaction behavior should specify only desired interaction dynamics; physical feasibility should be realized independently by the robot.

A pHRI software stack has two conceptually different responsibilities:

1) specify the desired interaction behavior; and
2) realize that behavior through commands compatible with the physical robot.

Architectural hypothesis. We argue for a general software-engineering principle, stated above, and for a specific reason it should hold: desired interaction behavior is a statement in interaction-state space — position error, velocity error, and human force — while physical feasibility is a statement in robot-constraint space — joint torque, joint limits, and workspace geometry. These are different mathematical objects with different natural variables, so a representation that mixes them (as a fixed impedance or PD law does) couples two things that do not have to be coupled. Separating them into a behavior layer and a realization layer connected by an explicit interface is therefore not only a convenient software boundary but a decomposition along the actual variables each responsibility depends on. This is an architectural claim, not merely an empirical one: it also implies that a behavior specification can be authored, tested, and replaced independently of the robot, and that feasibility logic can be verified once and reused across every behavior that respects the interface (Section VII argues this directly). This paper tests the principle with one executable desired-acceleration interface, developed and validated only for its memoryless affine subclass (Sections III-B–VI); that affine QP realization layer is one instance supporting the general architectural claim, tested here on its memoryless affine subclass.

Conventional impedance control combines behavior specification and command realization in one mass–spring–damper feedback law. Variable-impedance methods add adaptation, and model-predictive variants can optimize impedance parameters or jointly plan motion and compliance. Those formulations are useful, but changing the behavior representation generally changes the controller that interprets it.

A concrete consequence of that coupling is actuator saturation. A conventional impedance or PD law computes joint torque directly from the interaction error, $\tau = J^\top(-K_d e - D_d \dot{e}) + \tau_{\mathrm{ff}}$, where τ_{ff} includes gravity/Coriolis compensation and, on a redundant arm, orientation-hold and null-space terms. Neither term reasons about the actuator limit. A stiff impedance, large interaction force, or unfavorable configuration can therefore drive the command past $\tau_{\max}$, and clipping silently changes the delivered acceleration.

The architecture studied here assigns this conflict to a realization layer. Its current predictive implementation constrains the *total frozen-model torque* — feedforward, orientation, null-space, and interaction terms — at every prediction step (Section VI-A). Any deviation from the behavior specification appears in the total realization residual; a counterfactual audit distinguishes constraint intervention from secondary-objective and model terms. Because the manipulator prediction is frozen at each solve, this remains a local-model mechanism rather than a global nonlinear-plant guarantee.

We study a different decomposition:

The behavior layer expresses intent without choosing a robot command. The realization runtime handles feasibility, constraint enforcement, and command optimization without

Y. Cao is with Voryx Robotics LLC, San Jose, CA 95136, USA (e-mail: yongyancao@gmail.com).

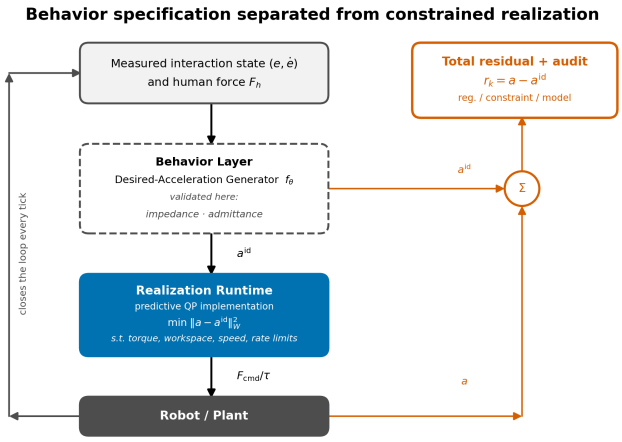

Fig. 1. Behavior and realization as separate layers. The behavior layer (dashed) supplies a^{id}; the realization runtime (blue) converts it into a constrained command and reports $r_k = a - a^{\mathrm{id}}$; the audit in (18) separates its causes.

deciding what stiffness, damping, or force-response semantics should mean. The runtime reports total behavior error and a constraint-intervention diagnostic rather than conflating the two.

The current executable instance. For memoryless affine behavior models, the predictive runtime retains one decision variable, feasible set, and objective template; the desired behavioral acceleration enters through (C_θ, G_θ). Section IV states this limited structural property formally, and Section III-B implements impedance and admittance. Nonlinear or stateful specifications may require different prediction machinery and are not instances of the present QP by assertion.

This separation changes the scientific question. Instead of asking whether a particular MPC formulation produces compliant tracking, we ask:

> *How should robot-control software separate desired interaction behavior from its constrained physical realization, and how should unavoidable intervention be exposed?*

The contributions are:

- a behavior–realization architecture that assigns behavior semantics and physical command feasibility to separate software layers;
- a precise desired-acceleration interface, a total realization residual, and a same-objective constrained/unconstrained audit that makes constraint intervention separately observable;
- one predictive realization runtime for the memoryless affine subclass, with a fixed robot-command variable and feasible set across impedance and admittance behavior layers;
- reproducible planar and FR3 simulations showing online behavior-layer replacement, anticipatory state-constraint handling, and manipulator-level realization under command and torque limits.

Task-space inverse dynamics, whole-body QPs, reference governors, and hierarchical robot controllers already separate related responsibilities, while model predictive impedance and interaction controllers provide important integrated formulations. The contribution is the four-part combination of (i) an interaction law evaluated along predicted states and exchanged at an acceleration-level contract, (ii) one fixed robot-command runtime across generators, (iii) a same-objective constrained/unconstrained counterfactual that decomposes the realization residual, and (iv) live generator substitution without rebuilding runtime state. The executable evidence covers two affine behaviors.

II. Related Work and Positioning

The closest literature is best distinguished by what the predictive or adaptive layer optimizes and by what quantity the robot-level controller is asked to reproduce.

Several approaches use MPC to adapt impedance parameters or to plan impedance and motion together. Anand et al. place a learned predictive policy above a low-level variable-impedance controller [1]. Haninger et al. predict interaction and optimize trajectory and impedance online while incorporating safety constraints [2]. Recent predictive variable-impedance formulations continue this parameter-adaptation direction [3]. Their optimized behavior variables are stiffness, damping, or related trajectory parameters.

A related line of work is model-reference adaptive impedance control, which specifies desired impedance dynamics and designs an adaptive nonlinear controller so that the robot approaches them [4]. This establishes the value of separating a desired model from the physical robot, but it does not by itself provide horizon-wide handling of coupled state and input constraints.

A separate class applies MPC directly to the robot or to a feedback-linearized error model, with decision variables that are robot commands rather than impedance parameters. Such a controller can generate an effective closed-loop impedance, but unless a desired interaction model appears explicitly in the prediction objective, that impedance is an induced property of the optimizer rather than the behavior it is asked to realize. This distinction separates the present formulation from our earlier double-integrator tracking MPC: the earlier controller penalizes interaction error, whereas the present controller penalizes disagreement with an independently specified interaction dynamical system.

Task-space inverse dynamics and hierarchical whole-body QPs are a more fundamental precedent for the interface itself: they already accept desired task accelerations as modular objectives and solve for dynamically feasible robot accelerations, contact forces, or torques. Del Prete et al.'s TSID, for example, treats prioritized motion/force tasks for constrained fully actuated robots [19]. Predictive hierarchical task-space control extends acceleration-error objectives over a finite horizon for underactuated and constrained robots [20]. Thus neither "desired task acceleration" nor "QP realization under robot constraints" is claimed as new. Relative to these frameworks, the contribution tested here is narrower: the task is an interaction dynamical law evaluated along the predicted state; the same runtime is retained across impedance and

TABLE I
POSITIONING AGAINST RELATED WORK.

Class	Behavior representation	Optimized robot-level variable	Role of prediction
Predictive variable impedance	M_d, D_d, K_d, often with a trajectory	Impedance parameters and/or trajectory	Select compliant behavior
Reference-model adaptive impedance	$M_d\ddot{e} + D_d\dot{e} + K_d e = F_h$	Adaptive feedback parameters	Usually absent
Direct robot MPC	State or tracking-error dynamics	Force, torque, velocity, or acceleration	Optimize robot motion directly
Impedance/interaction MPC	Impedance, force, or coupled interaction model	Robot command and sometimes behavior parameters	Embed interaction objectives and constraints
Reference governor	A single reference/setpoint signal	Filtered reference fed to a fixed inner-loop controller	Supervise a non-predictive inner loop to enforce constraints
Predictive safety filter	Nominal controller input	Minimally modified admissible input	Certify or replace a proposed command
Constrained compliance QP	Desired impedance/compliance response	Torque, velocity, or acceleration	Reproduce compliant behavior under robot constraints
TSID / hierarchical whole-body QP	Desired task accelerations and task priorities	Joint acceleration, contact force, and/or torque	Usually instantaneous constrained realization
Predictive hierarchical task-space control	Task trajectories and acceleration-level errors	Robot state and command trajectories	Extend hierarchical task objectives over a horizon
This work	$a^{\mathrm{id}} = C_\theta x + G_\theta F_h$	Robot command u	Minimize dynamics-realization error under constraints

admittance generators; its residual is split using a same-objective counterfactual; and the generator is replaced online without reconstructing the runtime state.

Closer still is model predictive impedance and interaction control. Bednarczyk et al. formulate impedance behavior through MPC and handle practical constraints such as velocity, energy, and jerk [5]. Gold et al. formulate model predictive interaction control using robot and interaction models in the optimal-control problem and express manipulation objectives through costs and constraints [6]. Minniti et al. combine whole-body MPC with online adaptation of robot–environment interaction models for constrained mobile manipulation [7]. These papers are close precedents and make a broad "first constrained interaction-dynamics MPC" claim untenable. Alharbat et al. explicitly compare NMPC impedance, cascaded admittance, and hybrid position/force paradigms [14]; their later predictive admittance controller predicts both robot and desired impedance states, respects actuator constraints, and is validated in physical aerial interaction [15]. Kumbhar et al. pass admittance-generated plans to a constrained whole-body QP on Digit hardware [17]. These are stronger validation precedents than the present simulation-only study; the remaining distinction sought here is the fixed cross-behavior desired-acceleration contract, not predictive compliance itself.

QP realization of compliant behavior and residual reporting also predate this work. Leboutet et al. compute robot commands under kinematic and dynamic constraints and propagate generalized-force residuals when a limb cannot realize the requested compliance [16]. More recent safety-critical adaptive impedance work filters a nominal impedance law through a QP with joint-state and torque constraints and provides formal invariance and boundedness results [18]. Accordingly, neither "QP-constrained compliance" nor "an explicit residual" is claimed independently as novel here.

Reference governors and predictive safety filters are the closest architectural relatives to this paper's separation, from a different literature than the parameter-adaptation and direct-MPC work described above. A reference governor sits upstream of a fixed, already-designed inner-loop controller and modifies the reference signal so that constraints on the resulting closed-loop trajectory are respected [8]. The parallel is direct: a reference governor also separates what is commanded from what is safe to execute. The difference is where prediction lives and what is exchanged. A reference governor typically supervises a fixed inner loop and manipulates a reference signal; the realization layer here is itself predictive, and the exchanged object is an affine desired-acceleration law evaluated over predicted interaction states. The realization residual additionally measures the difference between requested and delivered dynamics. Predictive safety filters similarly accept an arbitrary upstream command and use model-based backup optimization to certify or modify it [13]. This connection limits the novelty claim: the contribution is not separation in the abstract, but its formulation and measurement at the interaction-dynamics level.

Control barrier function (CBF) quadratic programs are another related architectural pattern from a different literature: a CBF-QP takes a nominal control input and projects it, at each tick, onto the minimal-deviation admissible set defined by a safety constraint [10]. The realization layer here is structurally

similar — a minimal-deviation projection of a requested behavior onto a robot-feasible set — but the projected object is a full behavior law evaluated over a receding horizon rather than a single nominal input. The total deviation is reported, and the constrained/unconstrained counterfactual identifies its constraint-intervention component rather than treating every nonzero residual as safety action. We do not compare against a CBF-QP filter or a reference-governor instance in this paper; Section VI-E lists a matched comparison against the closest alternative architecture as open.

Theorem 1 is a structural statement about a parametric quadratic program: for the affine generator class, the QP's decision variable, feasible set, and constraint structure are fixed while only its cost coefficients vary with the generator parameter θ. This is the object studied by the explicit MPC / multi-parametric QP literature [11], [12], which characterizes the solution map $\theta \mapsto U_k^\star(\theta)$ as piecewise affine. The present paper does not use that machinery — the QP is solved online at every tick, not precomputed offline — but Theorem 1 is best read as an instance of that established parametric-QP structure applied to the behavior–realization interface, not as a new structural fact about quadratic programs.

Most rows above optimize a quantity tied to one behavior representation: impedance parameters, adaptive impedance gains, or robot-level tracking against a specific interaction model. The reference-governor and safety-filter rows are architecturally closest. Our narrower distinction is an interaction-level behavior contract whose desired acceleration is separate from robot feasibility, together with a total residual and counterfactual intervention audit. The executable and theoretical evidence is limited to memoryless affine impedance and admittance behaviors.

III. Behavior–Realization Formulation

A. Behavior–Realization Interface

Consider a rigid manipulator

$$M(q)\ddot{q} + C(q,\dot{q})\dot{q} + g(q) = \tau + J(q)^\top F_h, \tag{1}$$

where $q \in \mathbb{R}^{n_q}$, $\tau \in \mathbb{R}^{n_q}$, and F_h is the human wrench applied at the interaction port. For a task coordinate $y = h(q) \in \mathbb{R}^{n_y}$,

$$\dot{y} = J(q)\dot{q}, \qquad \ddot{y} = J(q)\ddot{q} + \dot{J}(q,\dot{q})\dot{q}. \tag{2}$$

Substitution of the joint dynamics gives the affine acceleration map

$$\ddot{y} = b_y(q,\dot{q},F_h) + G_y(q)\tau, \tag{3}$$

where

$$b_y(q,\dot{q},F_h) = JM^{-1}\left(-C\dot{q} - g + J^\top F_h\right) + \dot{J}\dot{q}, \tag{4}$$

$$G_y(q) = JM^{-1}. \tag{5}$$

Let y_d be a nominal interaction pose and define

$$e = y - y_d, \qquad \dot{e} = \dot{y} - \dot{y}_d. \tag{6}$$

Definition 1 (Behavior-layer interface)**.** *In the present architecture, the behavior layer is a causal map*

$$\ddot{e}^{\mathrm{id}} = f_\theta(e,\dot{e},F_h,z), \tag{7}$$

where z denotes optional internal behavior state and θ denotes behavior parameters. Its output is a desired contact-port acceleration — termed the desired behavioral acceleration *throughout this paper — not a robot command. The behavior layer is deliberately unaware of robot torque limits, joint limits, or workspace geometry; those belong to the realization layer. The general map defines the interface boundary; the theorem and experiments below cover only its memoryless affine specialization.*

The architecture does not ask the behavior layer to compute τ. It supplies $\ddot{e}^{\mathrm{id}}$, and the realization layer selects τ so that the physical $\ddot{e}$ approaches $\ddot{e}^{\mathrm{id}}$ while satisfying robot constraints. The predictive QP developed below is one realization-layer implementation, not the definition of the layer itself.

The full manipulator formulation is the target architecture. The planar study below uses the following exactly discretized specialization so that behavior–realization separation can be isolated without kinematic or inverse-dynamics confounds: a planar point mass,

$$m_r\ddot{p}(t) = u(t) + F_h(t), \tag{8}$$

where $p \in \mathbb{R}^2$ is displacement from a nominal interaction pose, $u \in \mathbb{R}^2$ is commanded robot force, $F_h \in \mathbb{R}^2$ is measured human force, and $m_r > 0$ is the realized robot mass. Define

$$x = \begin{bmatrix} p^\top & v^\top \end{bmatrix}^\top, \qquad v = \dot{p}. \tag{9}$$

Under zero-order hold with period Δt,

$$x_{k+1} = Ax_k + B(u_k + F_{h,k}), \tag{10}$$

$$A = \begin{bmatrix} I & \Delta t I \\ 0 & I \end{bmatrix}, \qquad B = \begin{bmatrix} \frac{\Delta t^2}{2m_r} I \\ \frac{\Delta t}{m_r} I \end{bmatrix}. \tag{11}$$

The actual acceleration is

$$a_k = \frac{u_k + F_{h,k}}{m_r}. \tag{12}$$

For this planar plant, Definition 1 specializes with $x_k = [p_k^\top \; v_k^\top]^\top$ in place of $(e,\dot{e})$:

$$a_k^{\mathrm{id}} = f_\theta(x_k, F_{h,k}, z_k). \tag{13}$$

For the affine class considered throughout this paper,

$$a_k^{\mathrm{id}} = C_\theta x_k + G_\theta F_{h,k}. \tag{14}$$

The gap between what a behavior specifies and what the robot delivers is the *total realization residual*,

$$r_k = a_k - a_k^{\mathrm{id}}. \tag{15}$$

If $r_k = 0$, the robot exactly realizes the desired behavioral acceleration at that sample. Nonzero r_k alone, however, does not prove that a constraint caused the deviation: the finite force and force-rate weights can already shift the unconstrained optimum, and a physical plant can differ from the prediction model.

We therefore audit three terms. Let $U_k^{c\star}$ be the deployed constrained optimum and $U_k^{0\star}$ the counterfactual optimum of the *same* horizon objective at the same state, force forecast, and previous command after removing physical constraints.

Let a_k^c and a_k^0 be their first-step model accelerations, and let a_k^{emp} be the acceleration measured from the plant. Define

$$r_k^{\text{reg}} = a_k^0 - a_k^{\text{id}}, \qquad r_k^{\text{con}} = a_k^c - a_k^0, \tag{16}$$

$$r_k^{\text{mod}} = a_k^{\text{emp}} - a_k^c. \tag{17}$$

Then the empirical total error closes exactly as

$$r_k^{\text{emp}} = r_k^{\text{reg}} + r_k^{\text{con}} + r_k^{\text{mod}}. \tag{18}$$

Here r^{reg} contains secondary-objective compromise, r^{con} is the joint intervention of all enforced constraints, and r^{mod} contains frozen-model and acceleration-measurement error. The split does not attribute r^{con} to one individual active row; that would require a multiplier- or leave-one-constraint-out analysis. For the exact planar model $r^{\text{mod}} = 0$. The FR3 audit evaluates the first two terms at 50 Hz manager updates and obtains r^{mod} from the following 1 kHz plant sample.

All four quantities retain physical acceleration units. They are logging and causal-diagnostic signals, not normalized cross-generator scores; a generator-relative or energy-normalized metric would answer a different comparison question.

This definition is intentionally stronger than commanding a nominal impedance force and penalizing

$$\left\| u_k - u_k^{\text{imp}} \right\|_2^2. \tag{19}$$

The latter is controller-output tracking: it penalizes deviation from a nominal force command. $\|r_k\|_W^2$ is tracking too, but of a categorically different target — the *desired behavioral acceleration* the generator specifies, not a prior controller's output — so it compares two physical accelerations directly. In this paper that distinction is demonstrated for impedance and admittance, neither of which requires the QP to track a prior controller output.

B. Implemented Behavior Layers

The two behavior layers implemented below use the same QP template developed in Section III-C and differ only in (C_θ, G_θ), as formalized by Theorem 1. The first is an impedance generator, whose desired impedance is

$$M_d a^{\text{id}} + D_d v + K_d p = F_h. \tag{20}$$

For isotropic parameters,

$$a^{\text{id}} = -\frac{K_d}{M_d} p - \frac{D_d}{M_d} v + \frac{1}{M_d} F_h, \tag{21}$$

so

$$C_{\text{imp}} = \begin{bmatrix} -K_d M_d^{-1} I & -D_d M_d^{-1} I \end{bmatrix}, \qquad G_{\text{imp}} = M_d^{-1} I. \tag{22}$$

Unlike variable-impedance MPC (Section II), M_d, D_d, K_d are not decision variables in the present formulation; they describe the requested behavior.

The second is a force-guided admittance generator, which requests a force-dependent velocity:

$$T_a a^{\text{id}} + v = Y F_h, \tag{23}$$

or

$$a^{\text{id}} = -T_a^{-1} v + T_a^{-1} Y F_h. \tag{24}$$

It has no position-restoring term. After force release, velocity decays and the displaced position is retained. This provides a qualitatively different behavior through the same acceleration interface.

C. Predictive Realization Runtime

Section III-B supplies a^{id}. For a memoryless affine generator it is represented by C_θ, G_θ, producing one condensed-QP template whose numerical cost coefficients change with the generator while its decision variable and feasible set do not. At time k, let

$$U_k = \begin{bmatrix} u_{0|k}^\top & \cdots & u_{N-1|k}^\top \end{bmatrix}^\top. \tag{25}$$

The controller solves

$$\begin{aligned} \min_{U_k} \quad & \sum_{i=0}^{N-1} \Big(\|a_{i|k} - a_{i|k}^{\text{id}}\|_W^2 + \lambda_u \|u_{i|k}\|_2^2 \\ & \qquad + \lambda_{\Delta u} \|u_{i|k} - u_{i-1|k}\|_2^2 \Big) \\ \text{s.t.} \quad & x_{i+1|k} = A x_{i|k} + B(u_{i|k} + \hat{F}_{h,i|k}), \\ & a_{i|k} = (u_{i|k} + \hat{F}_{h,i|k})/m_r, \\ & a_{i|k}^{\text{id}} = C_\theta x_{i|k} + G_\theta \hat{F}_{h,i|k}, \\ & |u_{i|k}|_\infty \le u_{\max}, \\ & |u_{i|k} - u_{i-1|k}|_\infty \le \dot{u}_{\max} \Delta t, \\ & |p_{i+1|k}|_\infty \le p_{\max}, \\ & |v_{i+1|k}|_\infty \le v_{\max}. \end{aligned} \tag{26}$$

Only $u_{0|k}^\star$ is executed. $u_{-1|k}$ in the $i = 0$ rate penalty and rate constraint denotes the command actually applied at the previous solve, carried by the controller between calls (zero before the first solve), so both are well-defined from the very first call. The implementation uses the measured force held constant across the horizon, $\hat{F}_{h,i|k} = F_{h,k}$, rather than oracle knowledge of the scripted force. A learned or physics-based force predictor can be substituted without changing the generator interface.

The objective $\|a_{i|k} - a_{i|k}^{\text{id}}\|_W^2$ is tracking — of a categorically different target than a position or velocity trajectory. No desired position trajectory is constructed; the optimizer instead compares two accelerations evaluated at the predicted interaction state: the acceleration the constrained robot will produce and the desired behavioral acceleration the generator specifies. Position and velocity enter as generator states and as safety-constrained quantities, never as tracked references in their own right.

The optimization variable is the robot command sequence U_k, not the generator's parameters. The generator parameters θ are fixed during each experiment. Switching from impedance to admittance changes C_θ, G_θ, not the QP constraints or the command variable — stated formally as Theorem 1 (Section IV).

IV. Properties of the Affine Runtime Instance

The architecture is independent of the following QP property, which shows how cleanly the separation is realized for the present affine subclass. This section establishes template invariance, convexity, exact realization under idealized conditions, and one-step constraint inheritance, while Proposition 3 states the boundary of that inheritance at recursive feasibility.

Theorem 1 (Affine-Generator Template-Invariance Property)**.** *Fix the plant (A, B, m_r), horizon N, weights $(W, \lambda_u, \lambda_{\Delta u})$, and bounds $(u_{\max}, \dot{u}_{\max}, p_{\max}, v_{\max})$. There is a single map $(C, G) \mapsto Q(C, G)$ — a quadratic-program template depending only on the plant, horizon, weights, and bounds above, never on θ — such that, for every generator θ with affine law (C_θ, G_θ), the realization QP of Section III-C is exactly $Q(C_\theta, G_\theta)$. Consequently, for any two generators θ_1, θ_2, the instances $Q(C_{\theta_1}, G_{\theta_1})$ and $Q(C_{\theta_2}, G_{\theta_2})$ share the same decision variable U_k and the same feasible set, and differ only in the numerical objective coefficients contributed by (C_θ, G_θ), not in the decision-variable dimensions, constraint set, or objective template. Realizing a different affine generator therefore changes only the supplied (C_θ, G_θ). A nonlinear generator can make $a^{\mathrm{id}}_{i|k}$ nonlinear in U_k, so the residual cost is no longer guaranteed convex quadratic and the template Q does not apply without linearization.*

Proof. Every constraint in Section III-C's QP — the dynamics recursion, the acceleration definition, and the four box constraints — is written in terms of A, B, m_r, $u_{\max}$, $\dot{u}_{\max}$, $p_{\max}$, $v_{\max}$, U_k, and the resulting state/acceleration sequence; none references θ, C_θ, or G_θ. The feasible set is therefore a fixed polyhedron, identical for every generator. The only appearance of (C_θ, G_θ) anywhere in the problem is the term $a^{\mathrm{id}}_{i|k} = C_\theta x_{i|k} + G_\theta \hat{F}_{h,i|k}$ inside the primary cost, entering exactly once per horizon step as an affine substitution into $r_{i|k} = a_{i|k} - a^{\mathrm{id}}_{i|k}$. Because $x_{i|k}$ is itself independent of (C_θ, G_θ) and affine in U_k (established in Proposition 1's proof below), $a^{\mathrm{id}}_{i|k}$ is affine in U_k with (C_θ, G_θ)-dependent coefficients only, so $\|r_{i|k}\|^2_W$ is a convex quadratic in U_k whose Hessian and gradient contributions are determined entirely by (C_θ, G_θ); the secondary terms $\lambda_u \|u_{i|k}\|_2^2$ and $\lambda_{\Delta u}\|u_{i|k} - u_{i-1|k}\|_2^2$ do not involve (C_θ, G_θ) at all. Collecting these pieces defines $Q(C, G)$ as a function of (C, G) alone — the plant, horizon, weights, and bounds enter Q as fixed parameters, not arguments — and by construction $Q(C_\theta, G_\theta)$ is exactly the QP Section III-C poses for generator θ. □

This is why Section III-B's two implemented generators are checkable rather than aspirational: each supplies a different (C_θ, G_θ) to the same template Q. Without linearization, a nonlinear or stateful f_θ does not factor through a fixed (C, G) pair and lies outside the theorem.

Proposition 1 (Convexity for affine generators)**.** *If $W \succeq 0$, $\lambda_u \geq 0$, and $\lambda_{\Delta u} \geq 0$, then the finite-horizon problem in Section III-C is a convex quadratic program. It is strictly convex if the assembled Hessian is positive definite, including the common case $\lambda_u > 0$.*

Proof. The lifted state $x_{i|k}$ is affine in U_k by the recursion $x_{i+1|k} = Ax_{i|k} + B(u_{i|k} + \hat{F}_{h,i|k})$ unrolled from $x_{0|k}$. Hence $a_{i|k} = (u_{i|k} + \hat{F}_{h,i|k})/m_r$ and $a^{\mathrm{id}}_{i|k} = C_\theta x_{i|k} + G_\theta \hat{F}_{h,i|k}$ are affine in U_k, so $r_{i|k} = a_{i|k} - a^{\mathrm{id}}_{i|k}$ is affine in U_k and $\|r_{i|k}\|^2_W$ is convex quadratic since $W \succeq 0$. The regularization terms $\lambda_u \|u_{i|k}\|_2^2$ and $\lambda_{\Delta u}\|u_{i|k} - u_{i-1|k}\|_2^2$ are convex quadratic in U_k for $\lambda_u, \lambda_{\Delta u} \geq 0$, and a sum of convex functions is convex. Every listed constraint bounds an affine function of U_k in absolute value, so the feasible set is an intersection of half-spaces, hence a convex polyhedron. □

Proposition 2 (Exact unconstrained realization)**.** *Assume additionally that $W \succ 0$ (strictly positive definite — a stronger hypothesis than Proposition 1's $W \succeq 0$). Suppose an input sequence exists for which $r_{i|k} = 0$ over the horizon and no constraint is active. If the realization residual is optimized lexicographically before secondary effort objectives—or equivalently the secondary weights are zero—then every optimal primary solution realizes the reference dynamics exactly.*

Proof. The primary objective $\sum_i \|r_{i|k}\|^2_W$ is nonnegative and attains the value zero at the assumed feasible sequence, so its minimum over the horizon is zero. Under lexicographic priority (or zero secondary weights), any optimal solution must attain this minimum, so $\sum_i \|r_{i|k}\|^2_W = 0$, and since each term is nonnegative, $\|r_{i|k}\|^2_W = 0$ for every i. Because $W \succ 0$, a quadratic form $\|r\|^2_W$ vanishes only at $r = 0$; if W were only positive semi-definite, this step would give $r_{i|k} \in \ker W$ rather than $r_{i|k} = 0$, which is why the stronger hypothesis is needed here but not in Proposition 1. □

With finite (not zero) secondary weights, predictive realization's measured residual is not exactly zero even during unconstrained intervals (Table II), consistent with Proposition 2's zero-weight idealization. The reactive comparator instead solves a different, unconstrained algebraic inversion of the generator's instantaneous law, so its residual sits at numerical-solver tolerance (Table II) — a check that the underlying generator and dynamics model are implemented correctly, not itself an instance of Proposition 2.

Proposition 3 (One-step constraint inheritance)**.** *Assume the model and current human force are exact over the executed sample and the QP is feasible. Then $u^\star_{0|k}$, $p_{1|k}$, and $v_{1|k}$ satisfy their corresponding QP bounds. Re-solving at every sample yields constraint satisfaction by induction while feasibility is retained.*

This proposition establishes constraint satisfaction with retained feasibility under re-solving. A deployable recursive-feasibility guarantee additionally requires a terminal invariant set, soft constraints with a quantified fallback, or a backup safe controller, together with robust coverage of sudden within-sample force changes and model error, via constraint tightening.

V. Planar Architecture Validation

Section IV establishes what the QP guarantees in principle — convexity, exact realization when unconstrained, and one-step constraint satisfaction — and the boundary of that guaran-

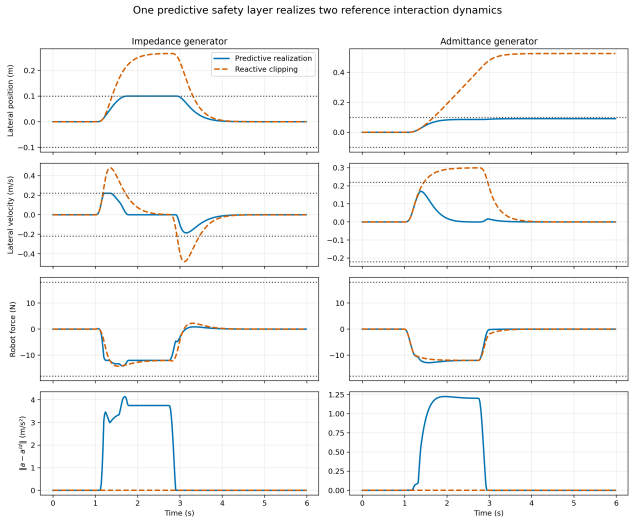

Fig. 2. Planar validation: impedance (left) and admittance (right). Dotted lines are state limits. Reactive clipping tracks the behavior but ignores the limits; predictive realization departs from it near the bound (final row).

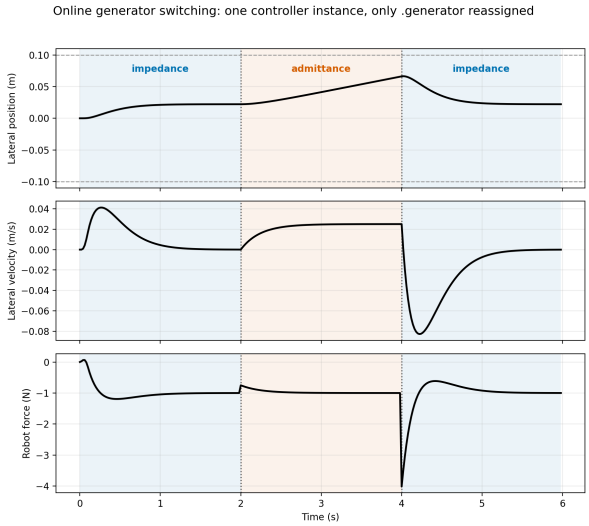

Fig. 3. One controller instance; only `.generator` is reassigned at $t = 2$ s and $t = 4$ s. Position converges to the impedance equilibrium in both impedance segments, drifts under admittance in between, and re-converges after re-entry from a different state.

tee at recursive feasibility. This section tests those properties empirically on the planar plant of Section III-A, comparing predictive realization against the reactive comparator under identical command and rate limits.

A. Reproducible Setup

The simulation uses a 2.5 kg planar point mass, $\Delta t = 0.02$ s, and a 20-step (0.4 s) horizon. A smooth 12 N force is applied along $+y$ from 1 s to 3 s. Limits are

$$|u_j| \le 18 \text{ N}, \qquad |v_j| \le 0.22 \text{ m/s}, \qquad |p_j| \le 0.10 \text{ m}, \quad (27)$$

with $|\dot{u}_j| \le 180$ N/s. The realization, force, and force-rate weights are 1, 2×10^{-4}, and 10^{-3}, respectively.

The impedance generator (Section III-B) uses

$$M_d = 2.0 \text{ kg}, \quad D_d = 18 \text{ Ns/m}, \quad K_d = 45 \text{ N/m}. \quad (28)$$

The admittance generator (Section III-B) uses $T_a = 0.25$ s and $Y = 0.025$ m/(Ns).

The comparator computes the instantaneous reference acceleration, converts it to robot force, and applies the same 18 N force and 180 N/s slew limits. It has no prediction of position or speed constraints. This comparator is intentionally minimal: it isolates what horizon-wide state constraints change.

B. Results

The impedance reference has a static displacement $F_h/K_d = 12/45 = 0.267$ m, matching the 0.266 m simulated peak after the smooth force ramp. Because this behavior is incompatible with the 0.10 m workspace, the predictive controller increases the realization residual while the bound is active. After release, it returns to the impedance equilibrium at the origin.

The admittance reference requests approximately 0.30 m/s under 12 N and has no restoring spring. Reactive realization therefore accumulates 0.525 m displacement and retains it after release. The constrained controller begins departing from the requested velocity dynamics before reaching the workspace boundary and settles at 0.091 m. The generator remains unchanged. Table III shows that the observed modification is predominantly constraint intervention rather than secondary regularization; the stronger "entirely constraint-induced" wording would be false because the regularization term is small but nonzero.

C. Online Generator Switching

The preceding two subsections validate the generator interface by running two separate simulations, one per generator — informative, but not yet a demonstration that a single running controller accepts a different generator without redesign. This section tests that directly. One `InteractionDynamicsMPC` instance is constructed once, with the impedance generator; a small constant lateral force (1 N, well inside every bound) is applied for the entire 6 s run, with no release. At $t = 2$ s and $t = 4$ s, exactly one attribute is reassigned on the live controller — `controller.generator` — switching impedance → admittance → impedance; nothing else (the QP weights, constraints, decision variable, or the `previous_command` state carried between solves) is touched or reconstructed.

Both switches are rate-bounded: the largest command-force jump at a switch instant is 3.02 N at $t = 4$ s, 84% of the QP's 3.6 N per-tick limit. This is a material command change that satisfies the same discrete rate constraint that bounds every other step. Position and speed stay within 0.067 m and 0.083 m of the origin throughout, well inside the 0.10 m / 0.22 m/s bounds. The experiment therefore validates the software-level generator interface and the feasibility of an online swap under the existing rate constraint. Optimal switching, hybrid stability, and continuity of the generator law are separate questions. The second impedance segment additionally shows re-convergence

TABLE II
PLANAR VALIDATION.

Generator	Controller	RMSE* (m/s^2)	Peak $\lvert p_j \rvert$ (m)	Peak spd. (m/s)	Peak $\lvert u_j \rvert$ (N)	Violation
Impedance	Predictive	1.351	0.100002	0.220	14.16	2×10^{-6} m
Impedance	Reactive	$< 10^{-6}$	0.2663	0.480	14.24	0.1663 m, 0.260 m/s
Admittance	Predictive	0.405	0.0912	0.169	12.88	none
Admittance	Reactive	$< 10^{-6}$	0.5250	0.300	12.00	0.4250 m, 0.080 m/s

*Realization RMSE, componentwise. State-limit violation reports the amount by which the 0.10 m / 0.22 m/s workspace/speed bound is exceeded. Realization RMSE for the reactive comparator is at numerical-solver tolerance because it directly inverts the generator's instantaneous law; predictive realization's nonzero residual reflects finite secondary weights and constraint-driven deviation, aggregated over the whole run (Proposition 2, Section IV). As in Table IV, RMSE is *component-wise*: all residual components and time samples are pooled into one mean before the square root.

TABLE III
PLANAR RESIDUAL DECOMPOSITION (M/S^2).

Generator	Total	r^{reg}	r^{con}	Closure
Impedance	1.351	0.0113	1.339	$< 9\times10^{-16}$
Admittance	0.405	0.00261	0.403	$< 3\times10^{-16}$

The first three numerical columns are component-wise RMSE and need not add as scalars; (18) closes samplewise. "Closure" is its maximum absolute component error. The planar model is exact, so $r^{\mathrm{mod}} = 0$.

from the different position and nonzero velocity left by the admittance segment without special-casing on re-entry.

D. What the Experiments Establish

The results support four limited claims:

1) two qualitatively different affine interaction models use the same constrained predictive implementation;
2) a constrained/unconstrained counterfactual separates constraint intervention from the small regularization component of total behavior error;
3) horizon-wide state constraints prevent violations that command clipping alone cannot prevent in this deterministic model;
4) a single controller instance accepts a different generator online, mid-run, by reassigning one attribute, with its command change bounded by the existing rate constraint.

These establish the software-boundary and substitution claims; manipulator-level feasibility is taken up in Section VI, and coupled human–robot stability, passivity, robustness to force-estimation delay, and embedded real-time performance in Section VIII.

VI. FR3 RUNTIME VALIDATION

Section V isolates behavior–realization separation from kinematic and inverse-dynamics confounds by using a point mass. This section deploys the same interface on a torque-controlled 7-DOF Franka FR3 simulated in MuJoCo, replacing the exact double-integrator plant with the nonlinear, configuration-dependent manipulator dynamics of Section III-A.

A. Architecture

At each solve, let $J_v(q) \in \mathbb{R}^{3\times7}$ be the translational Jacobian and $\Lambda(q)^{-1} = J_v M(q)^{-1} J_v^\top$ the (regularized) inverse operational-space inertia for the position coordinate.

Full closed-loop law. Let R_d be the held reference orientation, $e_R(q) \in \mathbb{R}^3$ the corresponding orientation error, $J_w(q) \in \mathbb{R}^{3\times7}$ the rotational Jacobian, and q_0 a fixed neutral configuration. The commanded joint torque is assembled from a feedforward term τ_{ff}, a raw orientation-hold term τ_{orient}, a raw null-space centering term τ_{null}, and their projection τ_{aux} through the null-space projector $\bar{N}_v$:

$$\begin{aligned}
\tau_{\mathrm{ff}}(q,\dot q) &= C(q,\dot q)\dot q + g(q),\\
\tau_{\mathrm{orient}}(q,\dot q) &= J_w(q)^\top\big(-K_{\mathrm{rot}} e_R(q) - D_{\mathrm{rot}}\omega\big),\\
\tau_{\mathrm{null}}(q,\dot q) &= -k_{\mathrm{null}}(q-q_0) - d_{\mathrm{null}}\dot q,\\
\bar N_v(q) &= I_7 - M(q)^{-1}J_v(q)^\top\Lambda(q)\,J_v(q),\\
\tau_{\mathrm{aux}}(q,\dot q) &= \bar N_v(q)^\top\big[\tau_{\mathrm{orient}}(q,\dot q) + \tau_{\mathrm{null}}(q,\dot q)\big],\\
\tau_{\mathrm{base}}(q,\dot q) &= \tau_{\mathrm{ff}}(q,\dot q) + \tau_{\mathrm{aux}}(q,\dot q),
\end{aligned} \tag{29}$$

and the executed torque is

$$\tau = \tau_{\mathrm{base}}(q,\dot q) + J_v(q)^\top F_{\mathrm{cmd}}. \tag{30}$$

Task acceleration obeys $\ddot y = J_v\ddot q + \dot J_v\dot q$. Consequently, define

$$d_{\mathrm{known}}(q,\dot q) = J_v(q)M(q)^{-1}\tau_{\mathrm{aux}}(q,\dot q) + \dot J_v(q,\dot q)\dot q, \tag{31}$$

which contains both the projected auxiliary-torque contribution and the task-kinematic term. The residual translational dynamics used by the QP are

$$\ddot e = \Lambda(q)^{-1}\big(F_{\mathrm{cmd}} + F_h\big) + d_{\mathrm{known}}, \tag{32}$$

where the realized acceleration is *positive* in the commanded force, matching the point-mass convention of Section III-A. $F_{\mathrm{cmd}} \in \mathbb{R}^3$, the QP's Cartesian correction force, is supplied by whichever controller is active. The predictive controller sets $F_{\mathrm{cmd}} = F^\star_{0|k}$, the first element of the receding-horizon solution (Section III-C, condensed later in this section). The reactive comparator instead sets

$$F_{\mathrm{cmd}} = \mathrm{clip}\Big(\Lambda(q)\big[a^{\mathrm{id}}(x,F_h) - d_{\mathrm{known}}\big] - F_h\Big), \tag{33}$$

rate- and magnitude-limited to the same bounds as the predictive controller, with $a^{\mathrm{id}} = C_\theta x + G_\theta F_h$ the instantaneous desired behavioral acceleration (Section III-B). Both controllers share the identical τ_{base} and torque-assembly step; they differ

only in how F_{cmd} is chosen. Thus the manipulator realization map changes, but the generator interface does not.

$\bar{N}_v(q)$ is a dynamically consistent projector for the translation task [9]. Projecting both the orientation and null-space auxiliary torques through $\bar{N}_v(q)$, rather than leaving either unprojected, prevents them from leaking into $\ddot{e}$ and inflating joint drift and the compensating Cartesian command. Because Λ uses Tikhonov regularization, $J_v M^{-1} \bar{N}_v^\top = 0$ holds only approximately; the remaining leakage and $\dot{J}_v \dot{q}$ are therefore retained in d_{known}.

A six-second gain sweep over all four conditions (`sweep_null_space_gains.py`) exposed the remaining slow drift. With $k_{\text{null}} = 10, d_{\text{null}} = 2$, configuration deviation reaches 1.32–1.85 rad and the reactive impedance condition exceeds a joint limit by about 1.6 N·m. Gains 40/8 are the gentlest tested values that remove this violation and keep deviation below 0.29 rad; they are used in the benchmark. They also alter reactive task-space peaks (admittance 0.474 m to 0.107 m; impedance 0.158 m to 0.104 m), whereas predictive peaks stay near the 0.06 m workspace bound. Gains 100/20 reduce drift below 0.19 rad but suppress admittance further, including its predictive peak to 0.044 m. The selected gains therefore control drift while modifying task-space behavior, as the peak values above show.

The realization QP freezes $\Lambda(q)^{-1}$, $d_{\text{known}}(q, \dot{q})$, $\tau_{\text{base}}(q, \dot{q})$, and $J_v(q)$ at the current solve and holds them across the horizon. In compact form, its decision vector contains the Cartesian command sequence and nonnegative state slacks,

$$\begin{aligned} \min_{F_i, s_i^p, s_i^v} \sum_{i=0}^{N-1} \Big(& \|\ddot{e}_i - a_i^{\text{id}}\|_W^2 + \lambda_F \|F_i\|_2^2 \\ & + \lambda_{\Delta F} \|F_i - F_{i-1}\|_2^2 + \rho\big((s_i^p)^2 + (s_i^v)^2\big) \Big), \end{aligned} \tag{34}$$

$$\begin{aligned} \ddot{e}_i &= \Lambda_k^{-1}(F_i + \hat{F}_{h,i}) + d_{\text{known},k}, \\ |F_i| &\le F_{\max}, \qquad |F_i - F_{i-1}| \le \Delta F_{\max}, \\ |\tau_{\text{base},k} + J_{v,k}^\top F_i| &\le \tau_{\max}, \\ |e_i| &\le e_{\max} + s_i^p, \\ |\dot{e}_i| &\le v_{\max} + s_i^v, \qquad s_i^p, s_i^v \ge 0. \end{aligned} \tag{35}$$

This per-solve model is a convex QP and a local predictor of the nonlinear MuJoCo plant. The hard torque constraint is imposed at every predicted step, rather than only $i = 0$. It guarantees feasibility of the frozen-model command sequence returned by a successful solve; the future nonlinear trajectory is covered operationally by recomputing and monitoring executed torque at 1 kHz.

The torque bound applies to the *total frozen-model torque* $\tau_{\text{base}} + J_v^\top F_i$, because the base term can consume actuator budget before the interaction correction is added. A successful solve therefore satisfies the frozen-model limit by construction. If the desired behavioral acceleration requires more torque, the QP returns the closest feasible command and exposes the trade-off through r_k.

The workspace and speed boxes use nonnegative slacks with $\rho = 10^8$, because the frozen-Jacobian model cannot guarantee recursive feasibility for the nonlinear plant. Nontrivial slack occurs in 110 impedance and 186 admittance solves, with peak position slack of 0.077 mm (impedance) and 0.027 mm (admittance) and peak speed slack below 10^{-6} m/s in both conditions. This slack is smaller than Table IV's executed peak $|e_z|$ violation (0.0001 m impedance, 0.0001 m admittance) because the two measure different objects: slack is the QP's own relaxation of its frozen-model prediction, while the executed violation is measured against the real nonlinear MuJoCo trajectory, which the frozen model only approximates. The separate reactive fallback for solver infeasibility is implemented but is not exercised: every reported solve is feasible.

The fast control loop recomputes τ_{base}, the orientation term, and the null-space term from the current $(q, \dot{q})$ at 1 kHz; the QP is re-solved only every 20 ms (a 50 Hz outer loop), holding F_{cmd} between solves. Caching the full joint torque at the outer-loop rate, rather than only the slow correction term, would confound any comparison of predictive against reactive control at a common inner-loop rate.

B. Benchmark Scenario

Mirroring the planar prototype's own story rather than a separate benchmark design: the EE holds a fixed nominal pose and orientation while a sustained human push toward a workspace/speed boundary is applied, and predictive realization is compared against a command/rate-limited reactive controller with no predictive workspace handling, under an identical command/rate box. Neither controller has oracle knowledge of the scripted force; both receive a zero-order-hold forecast of the currently measured force, as in Section V.

The push is a 20 N force along $-z$ with a raised-cosine ramp from $t = 1.0$ s to 1.25 s, a hold to $t = 3.25$ s, and a symmetric ramp-down to $t = 3.5$ s. It is applied physically to the MuJoCo end-effector body, not only supplied to the controller's model.

The 2 s hold gives the admittance velocity time to approach steady state ($T_a = 0.3$ s), which is needed for the reactive comparator to approach the workspace bound and so yield a predictive-versus-reactive contrast for admittance. The impedance peak, determined by its equilibrium, is reached within 0.5 s regardless of hold length.

The workspace/speed bound is

$$|e_z| \le 0.06 \text{ m}, \qquad |v| \le 0.35 \text{ m/s}, \tag{36}$$

the QP horizon is 15 steps (0.3 s) at $\Delta t = 0.02$ s, and per-joint torque limits match the FR3 (± 87 N·m for joints 1–4, ± 12 N·m for joints 5–7).

The impedance generator (Section III-B) uses $M_d = 2.0$ kg, $D_d = 28$ Ns/m, $K_d = 200$ N/m. The admittance generator (Section III-B) uses $T_a = 0.3$ s and $Y = 0.01$ m/(Ns). Each condition runs for 6 s of simulated time.

C. Results

No condition violates a per-joint torque limit at the executed sample, no solve is reported infeasible, and the torque constraint never binds in this primary benchmark: maximum

TABLE IV
FR3 BENCHMARK.

Generator	Controller	RMSE* (m/s^2)	Peak $\lvert e_z \rvert$ (m)	Peak spd. (m/s)	Torque util.	Violation
Impedance	Predictive	1.39 / 1.45	0.0601	0.300	37.0%	0.0001 m
Impedance	Reactive	0.223 / 0.062	0.1044	0.415	37.2%	0.0444 m, 0.065 m/s
Admittance	Predictive	0.212 / 0.289	0.0601	0.070	34.4%	0.0001 m
Admittance	Reactive	0.200 / 0.034	0.1066	0.041	31.7%	0.0466 m

*Residual RMSE, empirical / predicted. The *predicted* residual uses the frozen local model of Section VI-A. The *empirical* residual instead finite-differences the 1 kHz MuJoCo end-effector velocity and compares that acceleration with the generator request; it is the appropriate measure of behavior delivered by the simulated nonlinear plant. Their gap measures local-model and differentiation error; the predicted residual is a frozen-model quantity, distinct from the executed plant residual. As in Table II, both use component-wise RMSE: all three residual components and all time samples are pooled into one mean before the square root. This harmonizes the aggregation convention; cross-plant values are reported for consistency rather than as performance rankings. Speed is reported component-wise to match the box constraint. Maximum torque utilization is the largest ratio $|\tau_j|/\tau_{\max,j}$, which avoids comparing unlike 87 N·m and 12 N·m joint limits. Predictive solve times are reported in the text below because the reactive comparator does not solve a QP.

TABLE V
FR3 MANAGER-UPDATE RESIDUAL AUDIT (COMPONENT-WISE RMSE, M/S^2).

Generator	Model total	r^{reg}	r^{con}	r^{mod}	Empirical total	Model closure
Impedance	1.448	0.285	1.205	0.199	1.395	$< 5\times10^{-16}$
Admittance	0.283	0.0958	0.356	0.153	0.211	$< 3\times10^{-16}$

All columns use the 50 Hz manager-update instants; RMSE columns need not add because the vector terms can cancel. Model closure is the maximum absolute component error in $r^{\mathrm{model}} = r^{\mathrm{reg}} + r^{\mathrm{con}}$. r^{mod} then closes (18) against the finite-difference plant acceleration.

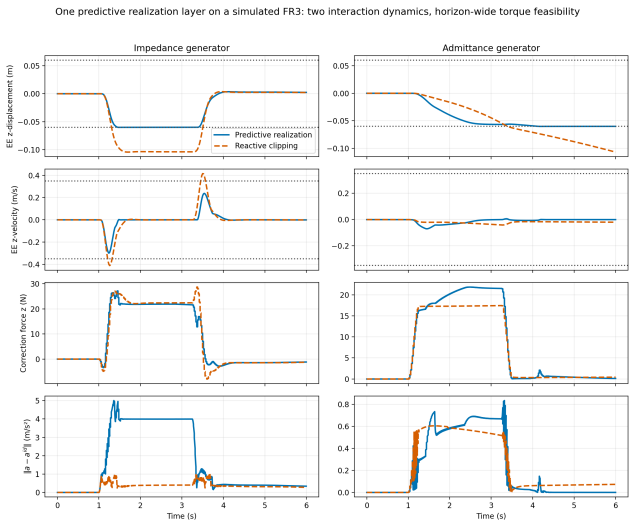

Fig. 4. FR3 benchmark: impedance (left) and admittance (right), predictive realization (solid) vs. the command/rate-limited reactive controller without predictive workspace handling (dashed). Dotted lines are workspace/speed bounds; bottom row is the empirical residual; torque does not clip in this primary benchmark. Predictive realization tracks the bound; the reactive controller overshoots and, for admittance, never recovers.

utilization stays below 38%. This benchmark therefore evaluates behavior realization at a workspace boundary, not torque intervention. Section VI-D introduces a deliberately derated torque-budget stress case in which the constraint activates.

An end-to-end unit test also tightens one joint limit until the horizon-wide constraint activates.

The impedance reference has an unconstrained static displacement $F_h/K_d = 20/200 = 0.10$ m, which by itself already exceeds the 0.06 m bound. Reactive clipping tracks toward that equilibrium and peaks at 0.104 m (slightly above the static value, from the same ramp-overshoot effect as the planar case in Section V-B), overshooting the bound by 4.4 cm. Predictive realization instead departs from the desired behavioral acceleration while the bound is active, holding displacement to 0.0601 m.

The admittance reference has no equilibrium displacement to compare against, only a steady-state velocity $YF_h = 0.01 \times 20 = 0.2$ m/s while the force is held, with no restoring term to pull it back after release. Reactive clipping accumulates displacement through the hold and continues briefly after force release before the velocity decays (time constant $T_a = 0.3$ s), peaking at 0.1066 m — a 4.7 cm overshoot — and never recovers it, since the generator itself has no position-restoring term (Section III-B) and the reactive comparator has no lookahead to anticipate the boundary. Predictive realization begins departing from the requested velocity before the bound is reached and stays within 0.0601 m.

Table V prevents the raw FR3 residual from being misread as constraint intervention alone. Constraint intervention is the largest component for both generators at manager updates, but finite regularization and plant/model terms are material, particularly for the admittance condition where vector cancellation makes the empirical total smaller than the individual component RMSE values.

The deployed QP uses the exact variable change $z = 10^3 s$ for each physical state slack s. Constraint coefficients use $z/10^3$, the slack Hessian weight becomes $\rho/10^6$, and reported values decode $s = z/10^3$; hence the physical objective and feasible set are unchanged. A unit test verifies this matrix transformation directly. It reduces the condensed Hessian condition number from 4.3×10^9 to 4.34×10^3 for impedance and from 3.8×10^9 to 3.77×10^3 for admittance. The deployed 50 Hz configuration also warm-starts OSQP from

the preceding primal/dual solution.

A dedicated timing study (`run_fr3_timing_study.py`, five full 6 s repetitions and 1500 actual solves per generator/condition) then measures the complete QP construction and solve. In the deployed warm-started configuration, impedance has 3.45 ms mean, 6.89 ms 99th-percentile, and 8.67 ms maximum solve time; admittance has 3.68, 7.28, and 9.73 ms, respectively. Thus all 3000 measured warm-started solves meet the 20 ms manager period on the development machine. The matched cold-start ablation has means of 4.24 and 4.02 ms, 99th percentiles of 7.25 and 8.33 ms, and maxima of 17.73 and 14.92 ms; all 3000 cold solves also meet 20 ms in the saved scaled-formulation study. Warm-starting is therefore retained for additional margin, rather than being credited as the sole deadline fix.

These wall-clock measurements are evidence for the tested machine; a hard real-time guarantee requires hardware-specific revalidation. They include QP construction and OSQP but exclude the post-solve unconstrained counterfactual used only for the residual audit. The simulator invokes the solve synchronously and does not implement asynchronous late-result discard; deployment on a different processor must re-run the timing test and, if overruns occur, hold the last applied command and reject stale solutions explicitly.

D. Torque-Active Runtime Intervention

The primary benchmark uses the nominal FR3 torque limits and does not activate them. To exercise runtime intervention without changing the behavior layer, we repeat the impedance condition with joint 4's available budget deliberately derated from its nominal 87 N·m to 31.5 N·m. This is an artificial actuator-budget stress test, not a claim about the FR3's physical rating. The 20 N force profile, impedance behavior, 0.3 s prediction horizon, QP weights, workspace/speed bounds, and 50 Hz solve rate remain unchanged.

Two predictive runtimes are compared. The proposed runtime constrains total frozen-model torque at all 15 predicted steps. The ablation constrains the same torque only at $i = 0$; its QP dimensions, behavior objective, state constraints, and command/rate bounds are otherwise identical.

This experiment tests the architectural responsibility assigned to the realization runtime: when a fixed behavior specification conflicts with an actuator budget, the runtime changes the command, exposes the resulting behavioral deviation, and avoids relying on an actuator-infeasible internal plan. The behavior layer itself is unchanged.

E. Scope and What Remains

This is a focused architecture validation. The implemented behavior layers are impedance and admittance; the scenario is a sustained push; the command/rate-limited reactive controller is a mechanism ablation. The online behavior-layer switching demonstration (Section V-C) is planar, with an FR3 counterpart a natural next step; sweeps over delay, sensor noise, and mass mismatch, and a collision constraint, extend the same runtime.

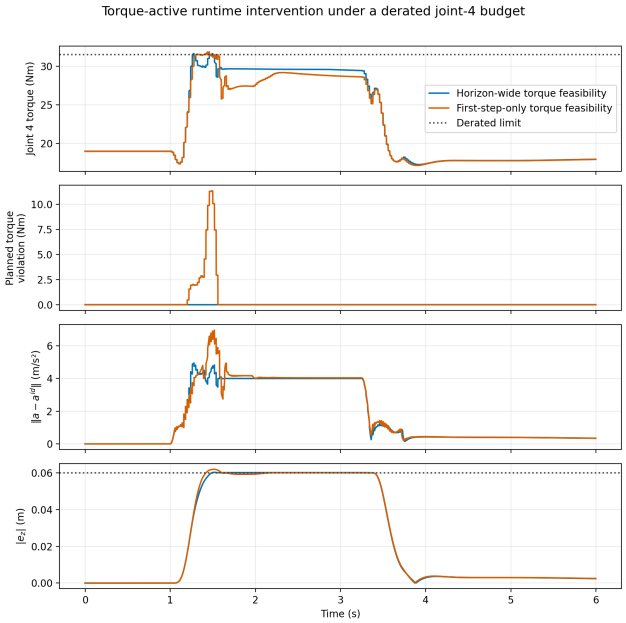

Fig. 5. Torque-active intervention under a derated joint-4 budget. Horizon-wide enforcement (blue) keeps the plan feasible; the first-step-only ablation (orange) satisfies only $i = 0$ and plans an infeasible future during the ramp. Dotted lines are the derated budget and workspace bound.

Further directions: human-participant and hardware validation; matched implementations of a predictive safety filter, predictive admittance controller, and MPIC baseline; and hardware-timing revalidation with an explicit stale-solution policy (the warm-started development-machine study meets 20 ms).

VII. Why Behavior–Realization Separation?

Interaction behavior should specify only desired interaction dynamics; physical feasibility should be realized independently by the robot. Section I argued this from the different variables each side depends on; Sections III-B–VI tested it through one affine executable instance. This section defends the separation directly, against three natural alternatives.

One alternative is to optimize behavior and robot commands jointly. Joint optimization is appropriate when behavior parameters are themselves task decisions, but separation instead makes a specification independently testable and replaceable while assigning feasibility to a stable downstream contract, at the cost that a fixed behavior layer may be suboptimal compared with a fully coupled design. This is an argument for a useful boundary.

A second alternative is direct robot MPC or trajectory tracking, which chooses commands to minimize a task objective or evaluates success against a position/velocity reference. The runtime here instead receives a law evaluated at the predicted interaction state and measures whether the robot reproduces the requested response to contact, with no trajectory generated; MPC is only the current mechanism, while the architectural choice is what the optimizer receives and what discrepancy it reports.

TABLE VI
TORQUE-ACTIVATION ABLATION.

Enforcement	Planned viol. (N·m)	Executed viol. (N·m)	RMSE* (m/s^2)	Peak $\lvert e_z \rvert$ (m)
All 15 steps	0.0002	0.161	1.398	0.0603
First step only	11.329	0.380	1.466	0.0619

*Empirical residual RMSE. The planned violation is evaluated against the frozen model used by each QP. The all-step value is numerical solver tolerance, whereas the first-step-only runtime plans future torque up to 11.329 N·m beyond the derated budget. Executed-sample torque is recomputed from the nonlinear MuJoCo state at 1 kHz; its smaller nonzero excess in both cases quantifies the local-model gap rather than a claimed hard nonlinear-plant guarantee. Horizon-wide enforcement reduces, but does not eliminate, that mismatch. The residual uses the component-wise convention of Tables II and IV.

A third alternative is a reference governor, which modifies a reference upstream of an already-designed fixed controller. The realization layer here is instead itself the command optimizer, and its input is a desired-acceleration law rather than a setpoint for a separate inner loop. Both separate intent from constraint handling; the distinguishing elements here are the explicit interface boundary, online cross-behavior substitution, and the decomposition of desired-versus-realized acceleration.

What separation buys, concretely, is controlled substitutability: Section V-C changes the behavior layer while preserving the realization object and its carried state, and Section VI-D changes the actuator budget while preserving the behavior — complementary tests of the same boundary, in which semantics can change without rebuilding feasibility logic, and feasibility can intervene without rewriting parameters.

VIII. LIMITATIONS AND OPEN THEORY

The results above establish the architectural distinction on two plants. Five directions extend it toward a mature behavior-realization framework.

First, the implemented behavior class is affine and memoryless; a stateful or learned model would need augmented prediction and explicit treatment of model uncertainty.

Second, the realization cost does not itself imply passivity — a passive reference generator can be rendered non-passive by constraint-induced deviation or secondary optimization. A dissipativity constraint on the realized port variables, paired with an energy tank or passivity observer, is a natural extension.

Third, hard state constraints can become infeasible after an unpredicted impulse (Proposition 3 identifies this as an open item). The FR3 study handles it operationally with slack-relaxed workspace/speed boxes and a reactive fallback on solver infeasibility; only the slack relaxation is exercised, and it stays small in the tested scenario. A deployable system additionally needs a terminal invariant set, a quantified soft-constraint risk, and a validated fallback path.

Fourth, the total realization residual has physical units and depends on the chosen output coordinates. Equation (18) separates regularization, joint constraint intervention, and model error; per-row intervention allocation and a normalized severity score are further refinements. The FR3 study exercises translation only; orientation is held by a fixed PD law, so extending the interface needs a principled or energy-weighted normalization across coordinates. Raw RMSE across generators with different scales is a diagnostic rather than a performance ranking, and Table IV's empirical and frozen-model residuals differ materially in several conditions, reflecting model-adequacy and estimation gaps within Theorem 1's frozen-model scope.

Finally, the contribution rests on implemented behavior-layer substitutability, formal residual guarantees, and matched experiments — demonstrated here for the affine memoryless subclass and positioned against the related work of Section II.

IX. CONCLUSION

This paper's contribution is architectural, not another interaction controller: desired-behavior specification and constrained robot realization are different responsibilities of robot-control software, made executable through a desired-acceleration interface and one predictive realization runtime for its memoryless affine subclass. The planar study swaps impedance for admittance and back while preserving the running realization object and its command state. The FR3 study instead changes the actuator budget while preserving the behavior layer: horizon-wide torque enforcement modifies the command, exposes the intervention through the constrained/unconstrained audit, and avoids the infeasible future plan a first-step-only ablation produces. Together these test both directions of the software boundary. The evidence is a reproducible proof of concept, scoped in Sections VI-E and VIII; future behavior layers may be physics-based, optimization-based, or learned only once they expose a compatible realization contract. Interaction behavior should specify only desired interaction dynamics; physical feasibility should be realized independently by the robot.

REPRODUCIBILITY

All experiments are driven by fixed, deterministic scenarios and are verified by an accompanying test suite that runs each scripted benchmark end to end and asserts its headline claims directly (torque-limit compliance, solver feasibility, workspace-bound margins, and the horizon-wide-versus-first-step-only comparison of Section VI-D). It also asserts sample-wise closure of the residual decomposition on the planar and frozen FR3 models rather than relying on inspection of a saved figure. Physical quantities (position, torque, and violation counts) are bit-for-bit reproducible from the fixed scenarios; solve times are wall-clock and are only ever bounded, never asserted exactly. The warm-start/Hessian-conditioning solve-time diagnosis of Section VI-C is produced by a dedicated

script (`run_fr3_timing_study.py`) separate from the main benchmark, with unit tests checking the exact slack-variable matrix transformation, confirming that warm-starting converges to the same command as a cold solve, and verifying that resetting a controller clears its carried warm-start state. Code, configuration, and saved numerical artifacts backing every figure and table in this paper are available in the project repository.

REFERENCES

[1] A. Anand et al., "Deep Model Predictive Variable Impedance Control," arXiv:2209.09614, 2022.

[2] K. Haninger, C. Hegeler, and L. Peternel, "Model Predictive Impedance Control with Gaussian Processes for Human and Environment Interaction," *Robotics and Autonomous Systems*, vol. 165, 104431, 2023.

[3] S. Xue et al., "Model predictive variable impedance control towards safe robotic interaction in unknown disturbance-rich environments," *Robotics and Autonomous Systems*, 2025.

[4] M. Sharifi, S. Behzadipour, and G. Vossoughi, "Nonlinear model reference adaptive impedance control for human–robot interactions," *Control Engineering Practice*, 2014.

[5] M. Bednarczyk et al., "Model Predictive Impedance Control," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2020.

[6] T. Gold, A. Völz, and K. Graichen, "Model Predictive Interaction Control for Robotic Manipulation Tasks," *IEEE Trans. Robotics*, 2023.

[7] M. V. Minniti, R. Grandia, K. Fäh, F. Farshidian, and M. Hutter, "Model Predictive Robot-Environment Interaction Control for Mobile Manipulation Tasks," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2021.

[8] E. Garone, S. Di Cairano, and I. Kolmanovsky, "Reference and command governors for systems with constraints: A survey on theory and applications," *Automatica*, vol. 75, pp. 306–328, 2017.

[9] O. Khatib, "A Unified Approach for Motion and Force Control of Robot Manipulators: The Operational Space Formulation," *IEEE J. Robotics and Automation*, vol. 3, no. 1, pp. 43–53, 1987.

[10] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, "Control Barrier Functions: Theory and Applications," in *Proc. European Control Conference (ECC)*, 2019.

[11] A. Bemporad, M. Morari, V. Dua, and E. N. Pistikopoulos, "The explicit linear quadratic regulator for constrained systems," *Automatica*, vol. 38, no. 1, pp. 3–20, 2002.

[12] P. Tondel, T. A. Johansen, and A. Bemporad, "An algorithm for multi-parametric quadratic programming and explicit MPC solutions," *Automatica*, vol. 39, no. 3, pp. 489–497, 2003.

[13] K. P. Wabersich and M. N. Zeilinger, "A predictive safety filter for learning-based control of constrained nonlinear dynamical systems," *Automatica*, vol. 129, Art. no. 109597, 2021.

[14] A. Alharbat, H. Esmaeeli, D. Bicego, A. Mersha, and A. Franchi, "Three fundamental paradigms for aerial physical interaction using nonlinear model predictive control," in *Proc. Int. Conf. Unmanned Aircraft Systems (ICUAS)*, 2022, pp. 39–48.

[15] A. Alharbat, C. Gabellieri, A. Mersha, and A. Franchi, "Predictive admittance control for aerial physical interaction," *IEEE Robot. Autom. Lett.*, vol. 10, no. 11, pp. 11235–11242, 2025.

[16] Q. Leboutet, E. Dean-Leon, F. Bergner, and G. Cheng, "Tactile-based whole-body compliance with force propagation for mobile manipulators," *IEEE Trans. Robotics*, vol. 35, no. 2, pp. 330–342, 2019.

[17] S. S. Kumbhar, A. M. Kulkarni, and P. Artemiadis, "Efficient and compliant control framework for versatile human–humanoid collaborative transportation," arXiv:2512.07819, 2025.

[18] F. Lawan, X. Han, J. Carrasco, B. Lennox, and X. Cheng, "Safety-critical adaptive impedance control via nonsmooth control barrier functions under state and input constraints," arXiv:2605.28367, 2026.

[19] A. Del Prete, F. Nori, G. Metta, and L. Natale, "Prioritized motion–force control of constrained fully-actuated robots: Task Space Inverse Dynamics," *Robotics and Autonomous Systems*, 2014, doi: 10.1016/j.robot.2014.08.016.

[20] J. Lee, S. H. Bang, E. Bakolas, and L. Sentis, "MPC-based hierarchical task space control of underactuated and constrained robots for execution of multiple tasks," arXiv:2009.05891, 2020.